a. The two GenAI programs I compared were Copilot and ChatGPT.
b. I accessed these programs through their respective sites: copilot.com and chatgpt.com
c. I used the same prompt for both GenAI programs to generate code:

--

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

• Use a fixed secret number (for example, 7).

• Use printf to prompt the user and scanf to read their guess.

• Use a while or for loop to let the user guess up to 3 times.

• Use if-else statements to check if the guess is too high, too low, or correct.

• Stop the loop immediately if the user guesses the correct number.

• Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

--

d. This resulted in two similar programs, but differing structures. The first is Copilot:

```
#include <stdio.h>

int main() {
  int secretNumber = 7;
  int guess;
  int attempts;
  int won = 0;

  printf("Guess a number between 1 and 10.\n");

  for (attempts = 1; attempts <= 3; attempts++) {
    printf("Attempt %d/3. Enter your guess: ", attempts);
    scanf("%d", &guess);

    if (guess == secretNumber) {
      printf("Correct! You win!\n");
      won = 1;
      break;  // Stop the loop immediately
    } else if (guess < secretNumber) {
      printf("Too low! Try again.\n");
    } else {
      printf("Too high! Try again.\n");
    }
  }

  if (!won) {
    printf("You lose! The secret number was %d.\n", secretNumber);
  }

  return 0;

}
```

And then ChatGPT's code:

```
#include <stdio.h>

int main() {
   int secret = 7;
   int guess;
   int attempt = 1;
   int correct = 0;

   printf("Guess a number between 1 and 10.\n");

   while (attempt <= 3) {
      printf("Attempt %d/3. Enter your guess: ", attempt);
      scanf("%d", &guess);

      if (guess == secret) {
         printf("Correct! You win!\n");
         correct = 1;
         break;
      } else if (guess < secret) {
         printf("Too low! Try again.\n");
      } else {
         printf("Too high! Try again.\n");
      }

      attempt++;
   }

   if (!correct) {
      printf("Sorry, you lose! The secret number was %d.\n", secret);
   }

   return 0;
```

}

The two generated code both perform the same tasks and share many similarities, but they approach the problem differently.

e. In terms of correctness, both programs fail to account for edge cases and contain no error handling. When the user inputs something other than an integer, the program silently crashes.

With the execution times of the two programs, since the ChatGPT generated code utilizes a while-loop instead of Copilot's for-loop, ChatGPT's code is slightly less optimized for time complexity due to for-loops being generally more efficient machine code.

Both ChatGPT and Copilot use the same amount of variables (5), however, one variable declaration could be removed if they did not have a win variable, but an else statement that would run after a for-else loop goes through all three attempts.

In terms of maintainability, both programs lack comments, such as a prologue and inline comments, and both fail to handle edge cases.

f. The code I chose for this assignment is Copilot because it uses for-loops instead of a while-loop while keeping the other parts of the code roughly the same, giving the code better time complexity.

g. To improve this code, the program considers edge cases like string input, integers outside of 1-10, and empty input. It also consider cases like an input of “123abc”, where an input of an int is followed by a char.

An improvement to both execution time and space complexity is moving the guess variables within the scope of the loop. This lets the integer be called whenever it is used, not just at the start of the program. This improves space by letting the var be declared when it is used and not at the start, and it improves execution time by letting the variable exist for less time, only for each loop iteration, instead of being initialized at the start of the program.

Maintainability is largely improved with the addition of prologue comments and inline comments for each line of code. It also handles edge cases now, avoiding random crashes.